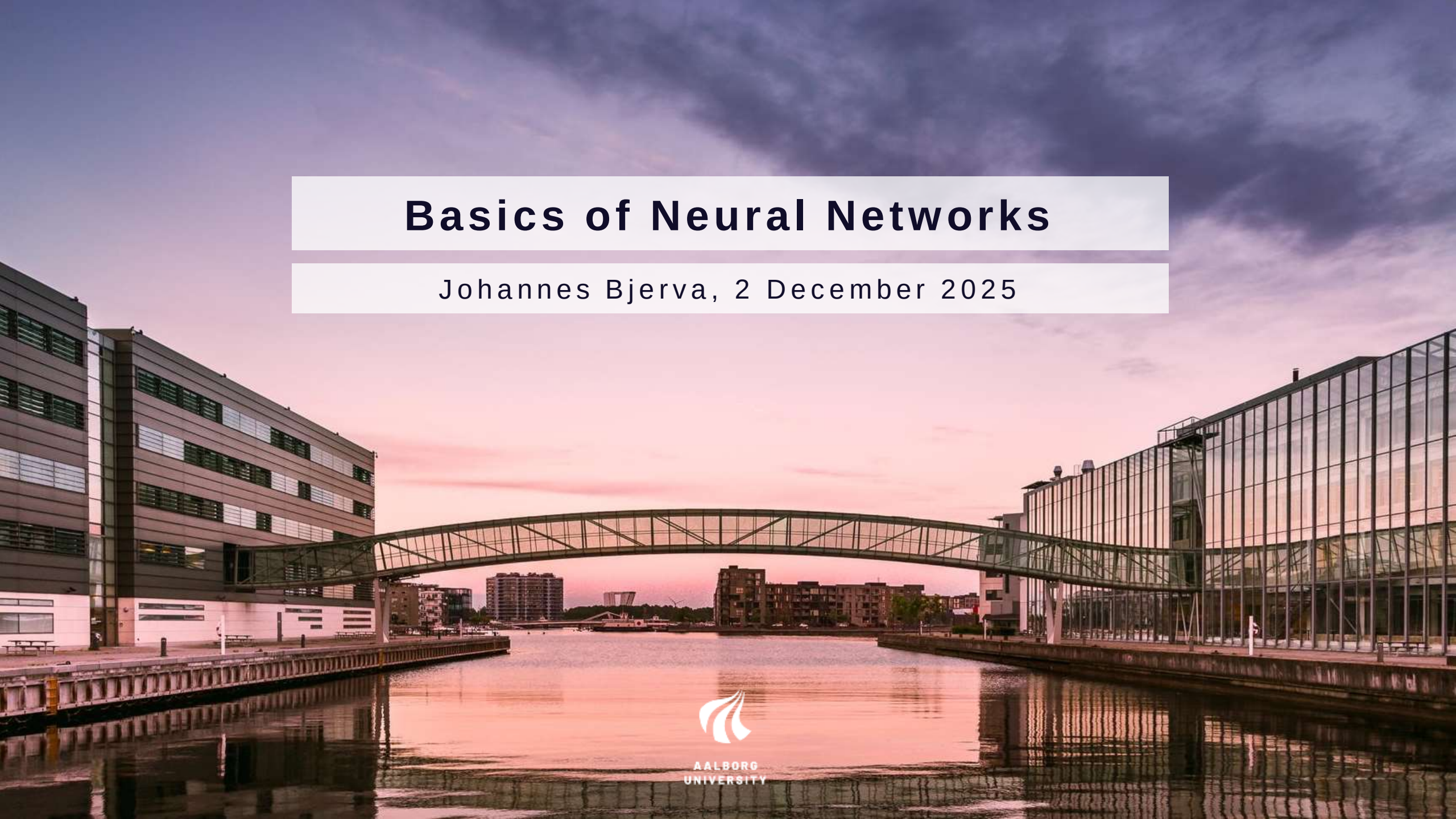
Basics of Neural Networks

Johannes Bjerva, 2 December 2025



AALBORG UNIVERSITY

About me:

- **Johannes Bjerva**
  - **Full Professor**
    Dept. of Computer Science, Aalborg University
  - **Ph.D. in Natural Language Processing** (NLP)
  - **~10 years of experience** in NLP
- Leading one of **DKs largest NLP Groups**
  - **Currently funded with ca. 25M DKK**
  - Interested in a research career? Talk to me about Research Assistant or PhD positions
- Live in Farum with my wife
  and three children (0.5, 2, and 4.5 years)

DANMARKS FRIE FORSKNINGSFOND
INDEPENDENT RESEARCH FUND DENMARK

novo nordisk foundation
Benefitting people and society

Innovationsfonden

VILLUM FONDEN

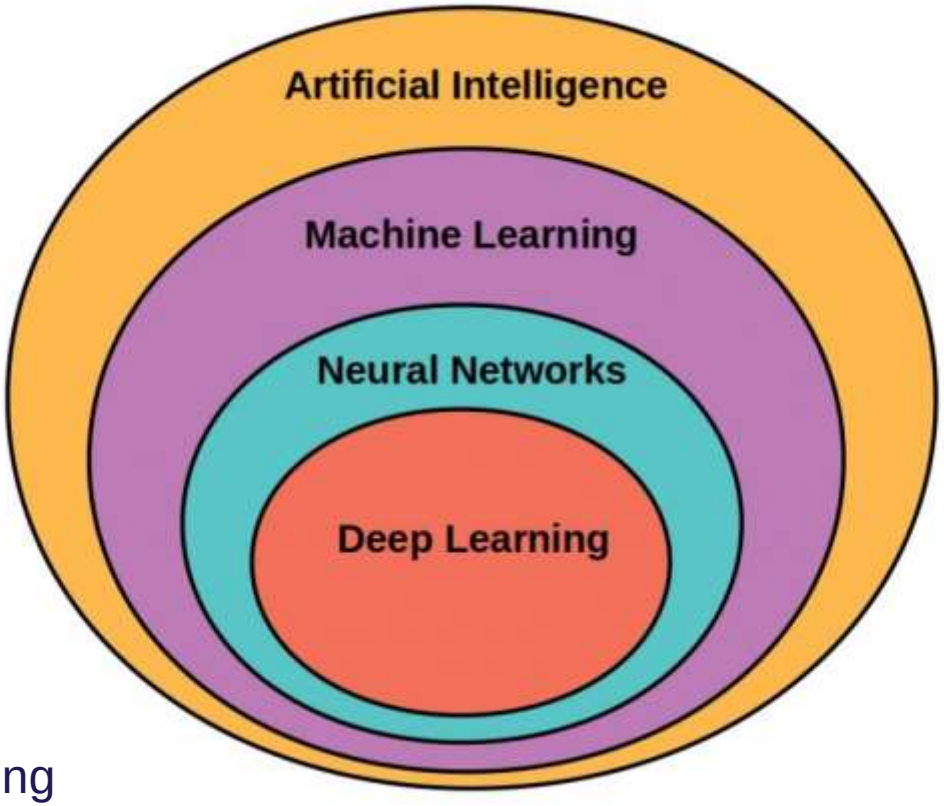
CARLSBERG FOUNDATION

Coefficient Giving

Google Research

Today's Overview

- Reading: Chapter 2 and Chapter 5 (partial)
  - *Deep Learning for Natural Language Processing: A Gentle Introduction* (Mihai Surdeanu and Marco A. Valenzuela-Escárcega)
- **Perceptrons**
- Introduction to Feed-Forward Neural Networks (**FFNNs**)
- **Implementation** of FFNN
- **The goal of today**:
  Lecture with theory mixed with implementation of Deep Learning



Learning Objectives

KNOWLEDGE

The student should gain knowledge of the following theories and methods:

- **problem solving using search and inference**
- model-based decision making
- inference under uncertainty
- learning from experience and **learning from data**

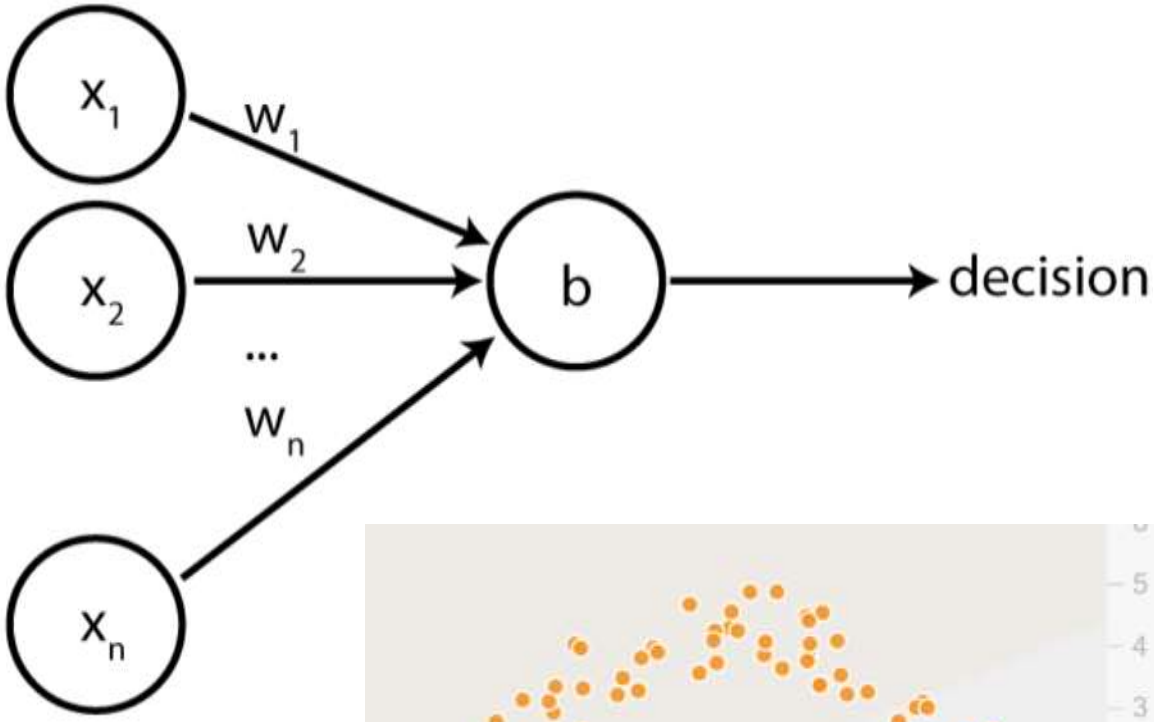
SKILLS

- use correct technical notation and terminology in writing as well as speech
- **apply basic techniques presented in the course to solve a specific problem**
- explain key principles and algorithms presented in the course

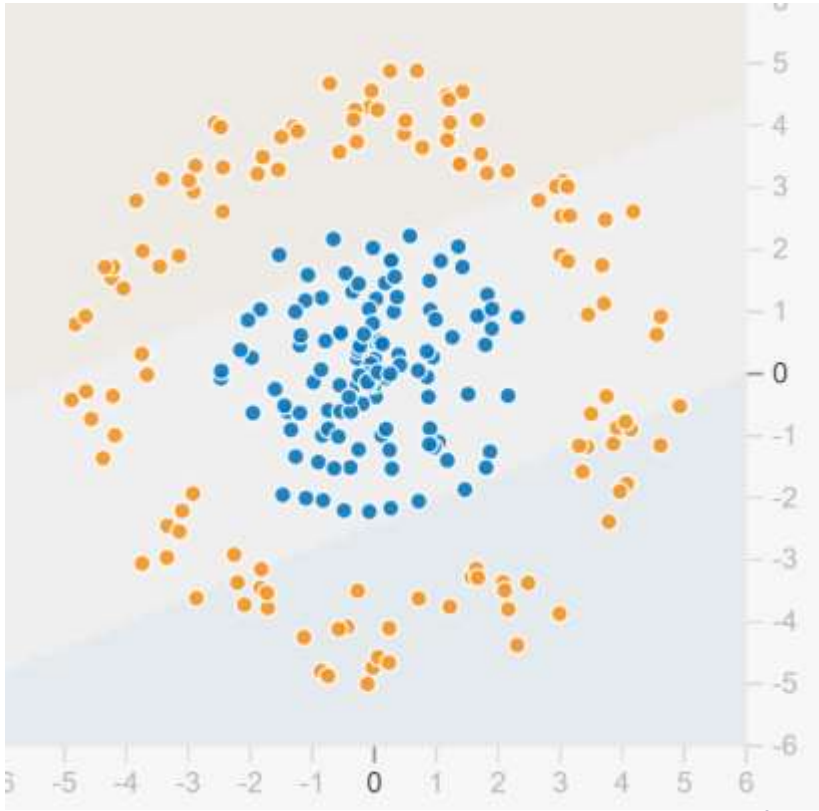
COMPETENCES

- be able to **evaluate**, compare and select techniques and methods within machine intelligence based on a specific problem

Perceptrons

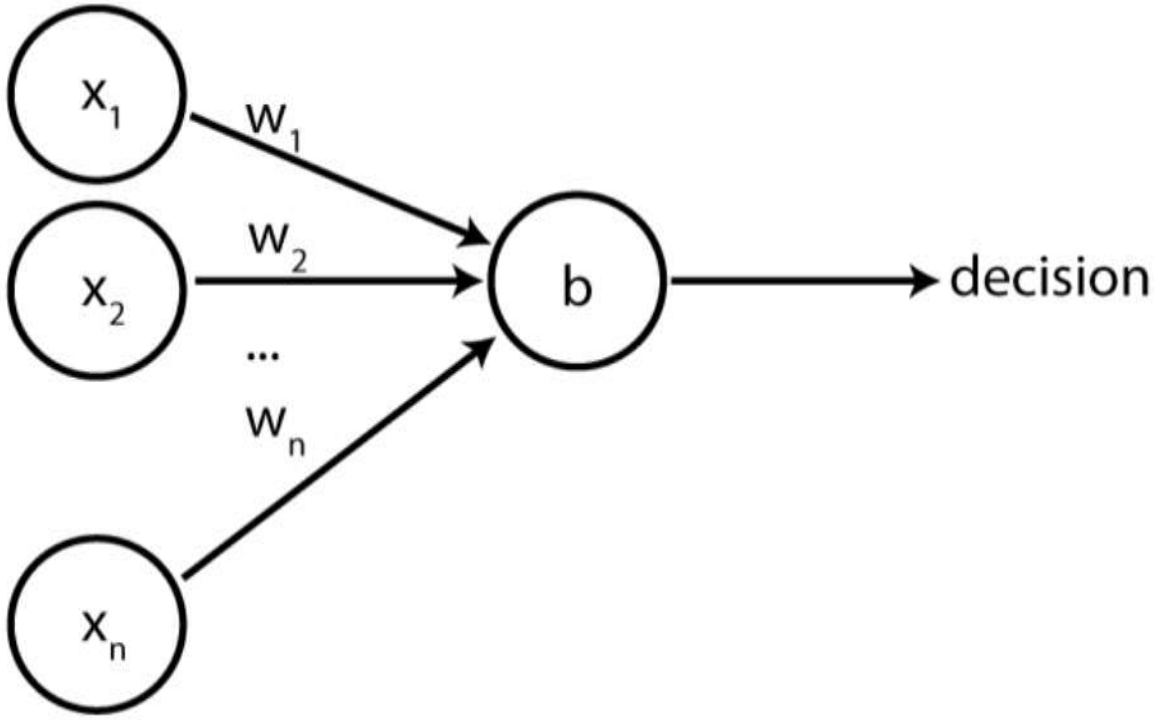


- Neural Network with **one** neuron
- Simple, but surprisingly effective
  - However, problems must be linearly separable!
- The basic building block for Neural Networks / Deep Learning



Inference in a Perceptron

$$x \cdot w = \sum_{i=1}^{n} (x_i \, w_i)$$

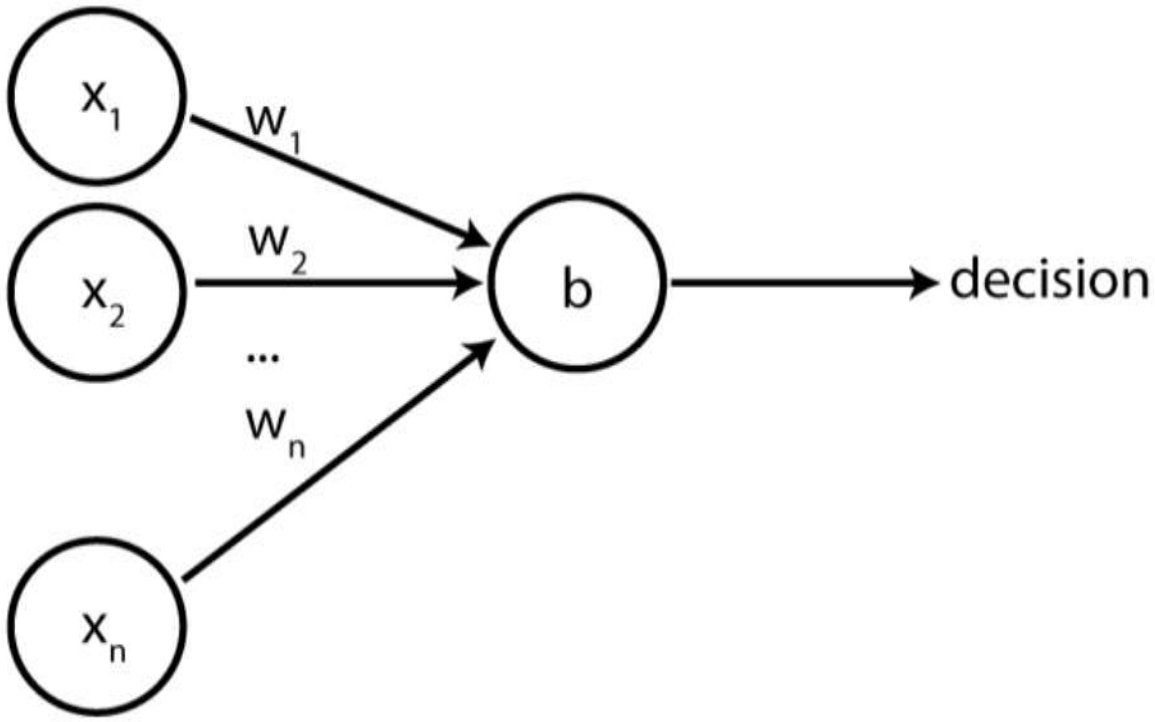


Algorithm 1: The decision function

```
if w · x + b > 0 then
    return Yes
else
    return No
end
```

Inference in a Perceptron

$$x \cdot w = \sum_{i=1}^{n} (x_i \, w_i)$$



Algorithm 1: The decision function

```
if w · x + b > 0 then
    return Yes
else
    return No
end
```

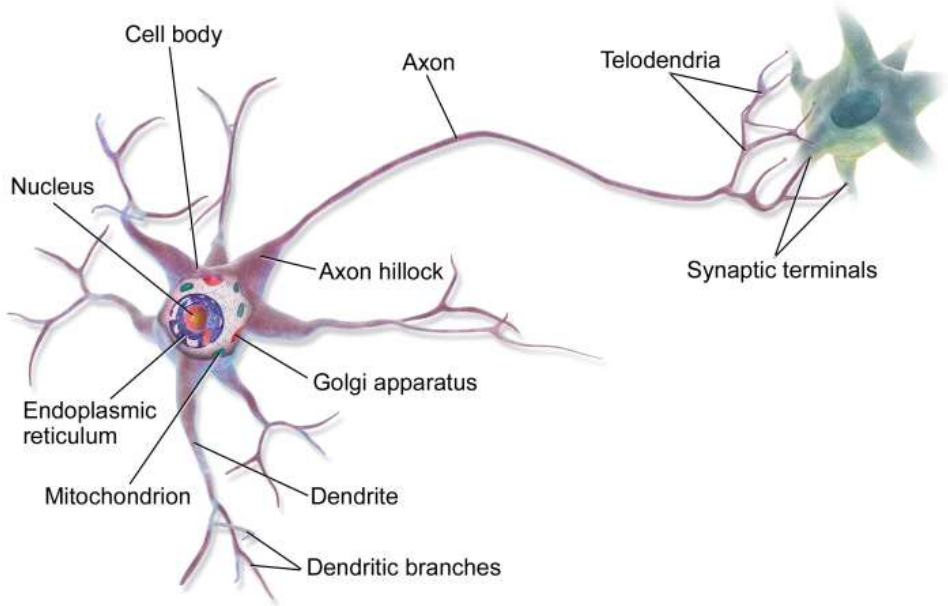
$$\mathbf{x} = (2, 1, 0)$$

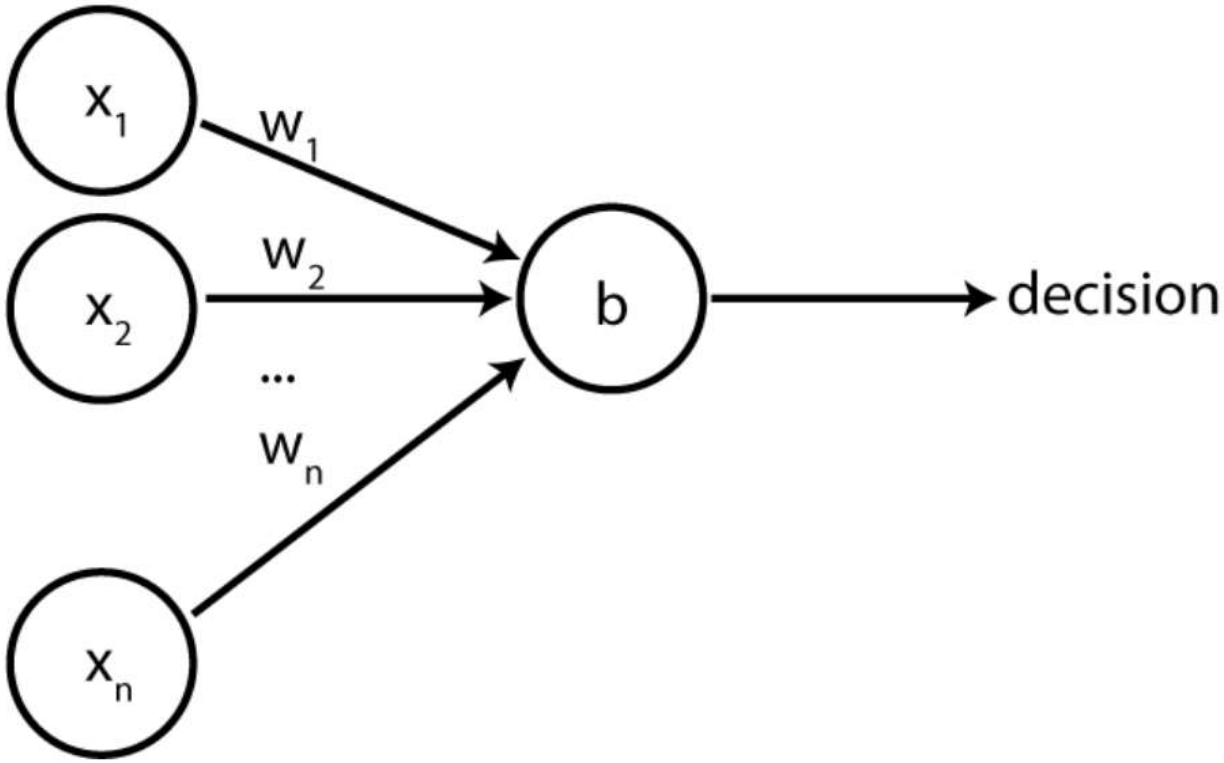
w = (5, -2, 3)

b = 1

Loosely based on real neurons (i.e. as in the brain)

- …but this is mainly **just inspiration**





Reminder:
Feature Representations

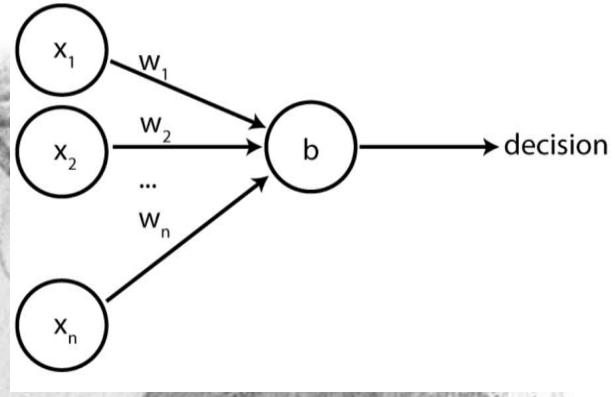


Table 2.1 *An example of a possible feature matrix* **X** *(left table) and a label vector* **y** *(right table) for three animals in our story: elephant, snake, and monkey.*

	isBig	hasTail	hasTrunk	hasColor Brown	numberOf Limbs
🐘					
🐍					
🐒					

Recap:
Feature Representations

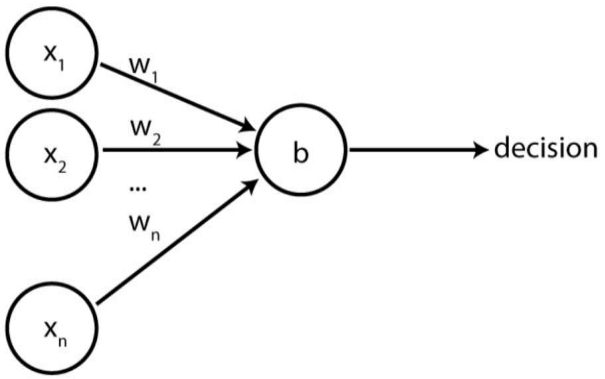


Table 2.1 *An example of a possible feature matrix* **X** *(left table) and a label vector* **y** *(right table) for three animals in our story: elephant, snake, and monkey.*

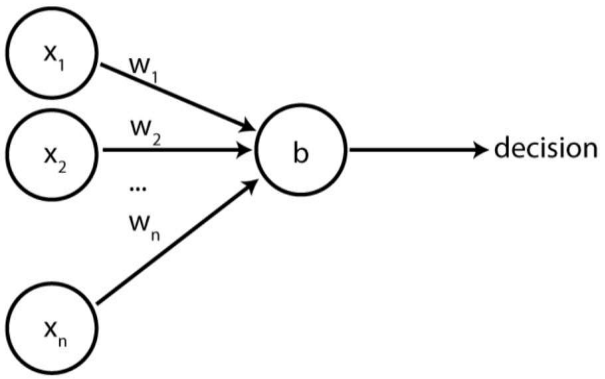
	isBig	hasTail	hasTrunk	hasColor Brown	numberOf Limbs
🐘	1	1	1	0	4
🐍	0	1	0	0	0
🐒	0	1	0	1	4

Notation:

- X: matrix of features
- y: vector of labels
- Each $y^{(i)}$ corresponds to $x^{(i)} \in X$

We want to feed an input x to a perceptron, and get an output ŷ

Inference –
Can we get the output we want?



	isBig	hasTail	hasTrunk	hasColor Brown	numberOf Limbs	Label
	1	1	1	0	4	isNotMyMom
	0	1	0	0	0	isNotMyMom
	0	1	0	1	4	isMyMom
W:	**-3**	**0**	**-3**	**3**	**2**	

$$x \cdot w = \sum_{i=1}^{n} (x_i \, w_i)$$

Assume b=1

```
if w · x + b > 0 then
    return Yes
else
    return No
end
```

We don't want to write these weights manually!

Perceptron learning

w – *vector* of weights

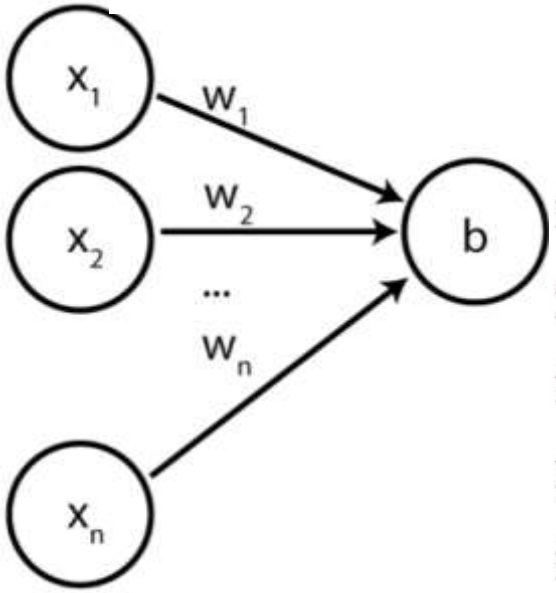
b – bias (int)

Prediction for each data point in the training set

If we're correct – great!

$$x \cdot w = \sum_{i=1}^{n} (x_i \, w_i)$$

Is it a **false negative?**



Is it a **false positive?**

Algorithm 2: Perceptron learning algorithm.

```
w = 0
b = 0
while not converged do
    for each training example x_i in X do
        d = decision(x_i, w, b)
        if d == y_i then
            continue
        else if y_i == Yes and d == No then
            b = b + 1
            w = w + x_i
        else if y_i == No and d == Yes then
            b = b - 1
            w = w - x_i
    end
end
```

Perceptron Learning example

isBig	hasTail	hasTrunk	hasColor Brown	numberOf Limbs	Label
1	1	1	0	4	isNotMyMom
0	1	0	0	0	isNotMyMom
0	1	0	1	4	isMyMom

w_1	w_2	w_3	w_4	w_5	b
1	1	1	1	1	1

decision($\mathbf{x}_i$, $\mathbf{w}$, b)

```
if w · x + b > 0 then
    return Yes
else
    return No
end
```

else if $y_i == Yes$ **and** $d == No$ **then**

$b = b + 1$

$\mathbf{w} = \mathbf{w} + \mathbf{x}_i$

False negative

else if $y_i == No$ **and** $d == Yes$ **then**

$b = b - 1$

$\mathbf{w} = \mathbf{w} - \mathbf{x}_i$

False positive

Implementation Time

- Work in **pairs** and implement the update part of the perceptron
- Pro-tip: ”Evaluate After” (ctrl+F10)

Think about / Experiment with:

- How is ”convergence” defined in the code?

- What happens if you:
  - Move the threshold?
  - Change the update criteria?
  - Remove the bias?

Answers in the form:
tinyurl.com/mi-e25-perceptron

Algorithm 2: Perceptron learning algorithm.

```
w = 0
b = 0
while not converged do
    for each training example x_i in X do
        d = decision(x_i, w, b)
        if d == y_i then
            continue
        else if y_i == Yes and d == No then
            b = b + 1
            w = w + x_i
        else if y_i == No and d == Yes then
            b = b - 1
            w = w - x_i
    end
end
```

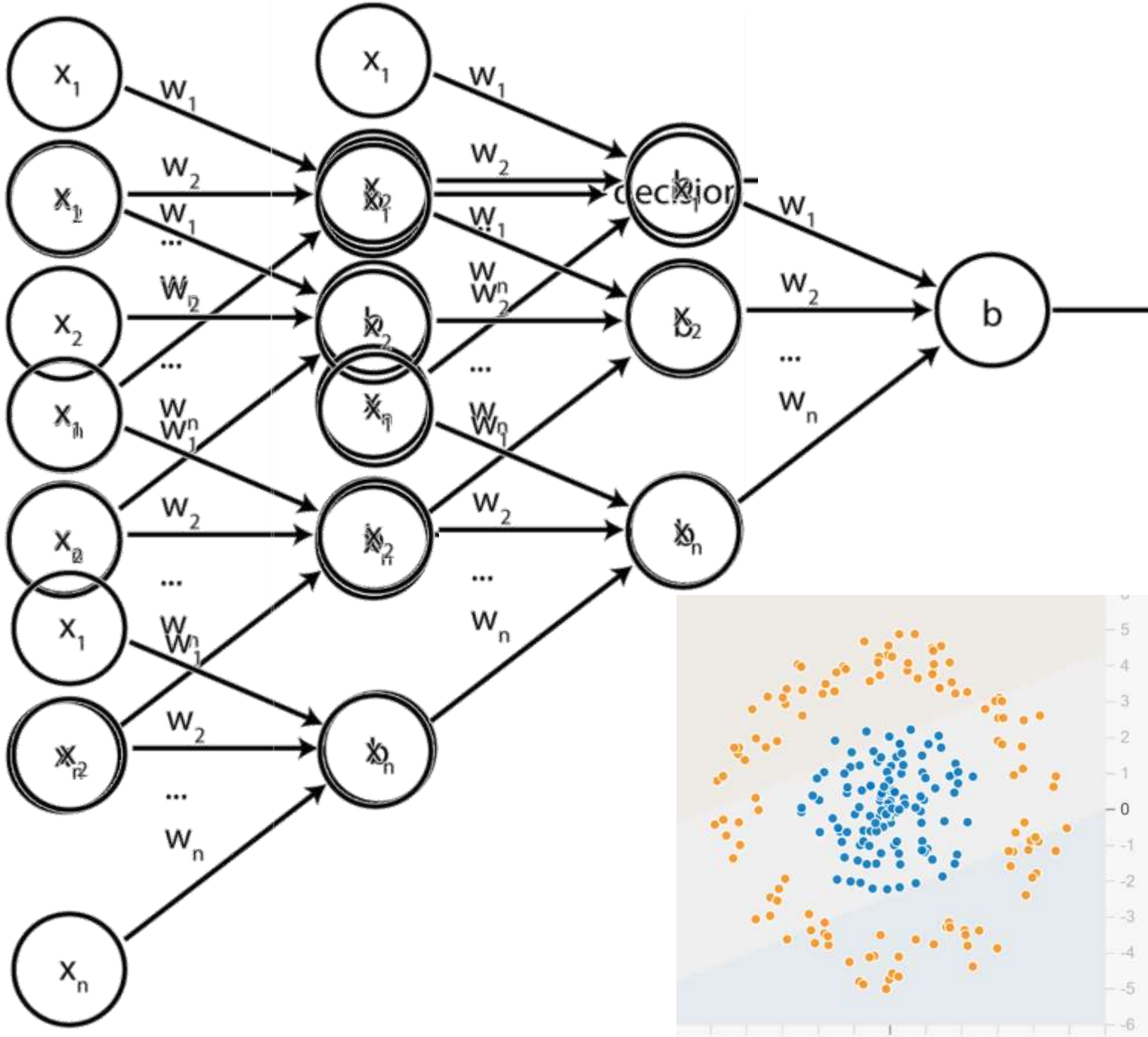
Pause til 14:30 (???)

Feed-Forward Neural Networks (FFNNs)

- One ”neuron” is limited to linear problems
- Most problems are *not* linear.

- …how about just using a lot more?

- Also known as an ”MLP”
  - Multi-Layer Perceptron



Tensorflow Playground

- Beyond Linearly Separable Problems
- https://playground.tensorflow.org/#activation=sigmoid&batchSize=10&dataset=xor&regDataset=reg-plane&learningRate=0.01&regularizationRate=0&noise=0&networkShape=3&seed=0.58065&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false

FFNNs: Terminology

- Input
- Hidden / Intermediate layers
- Weights
- Activations
- Output ('y-hat')

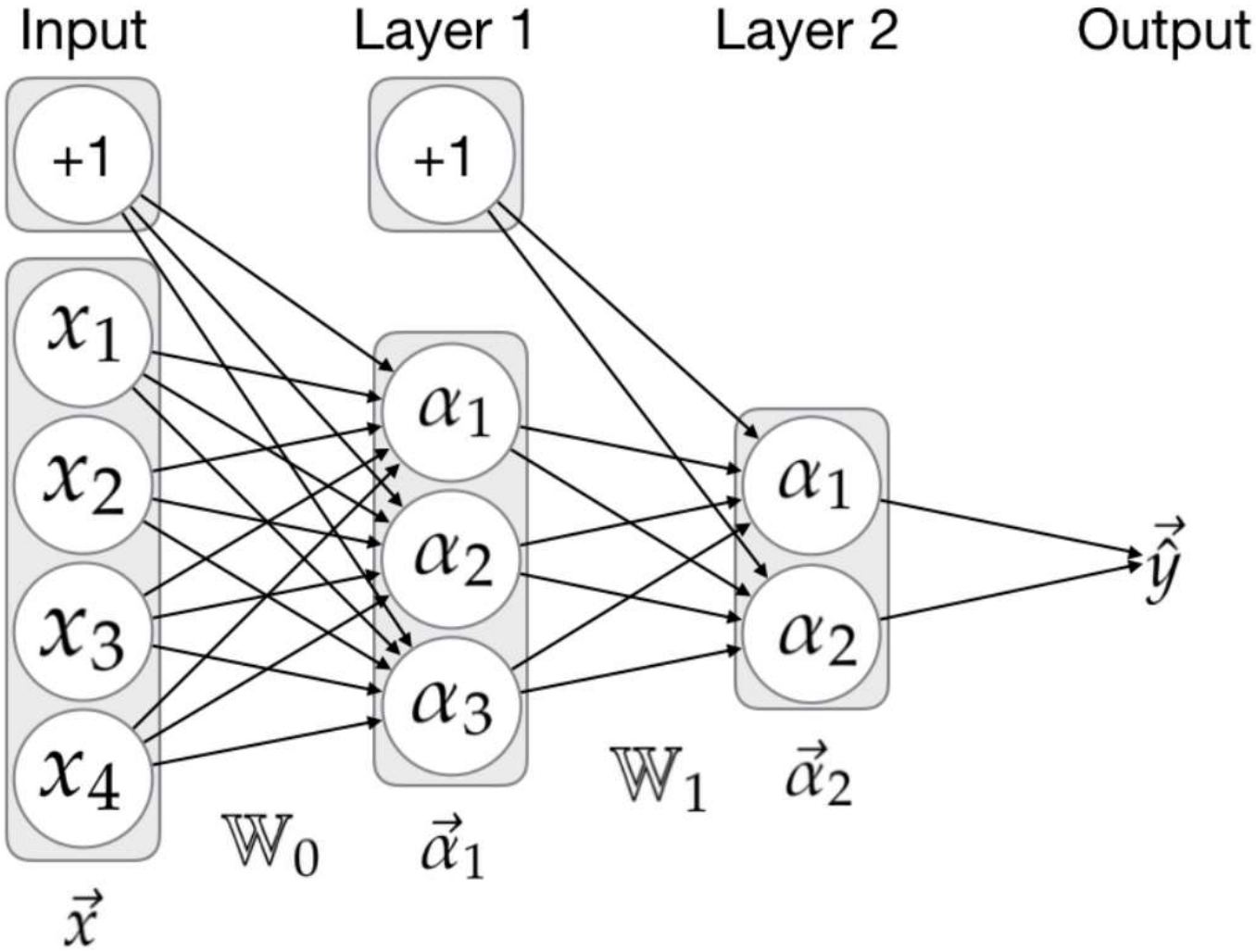
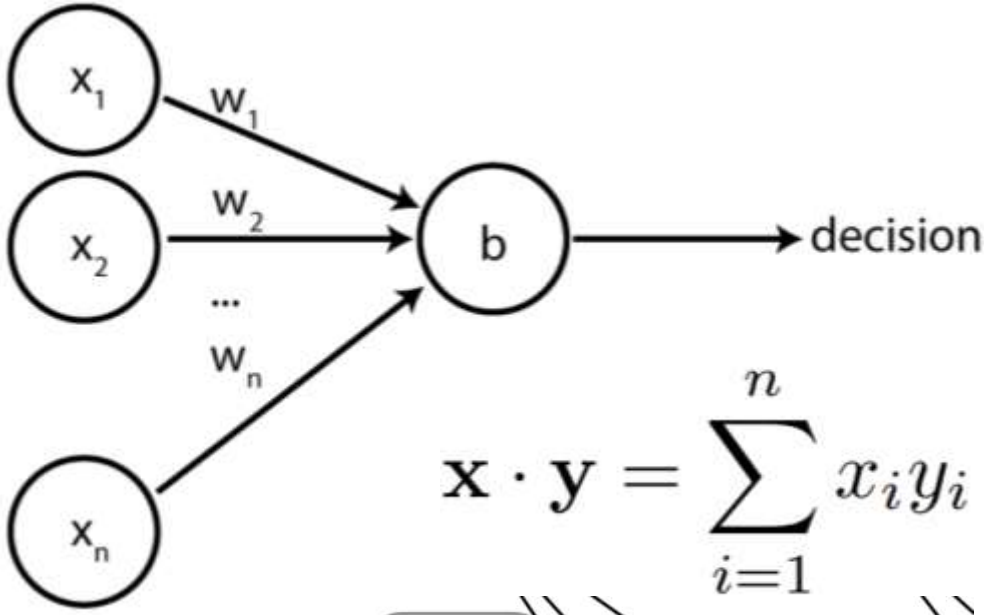


Figure 2.1: A basic Neural Network.

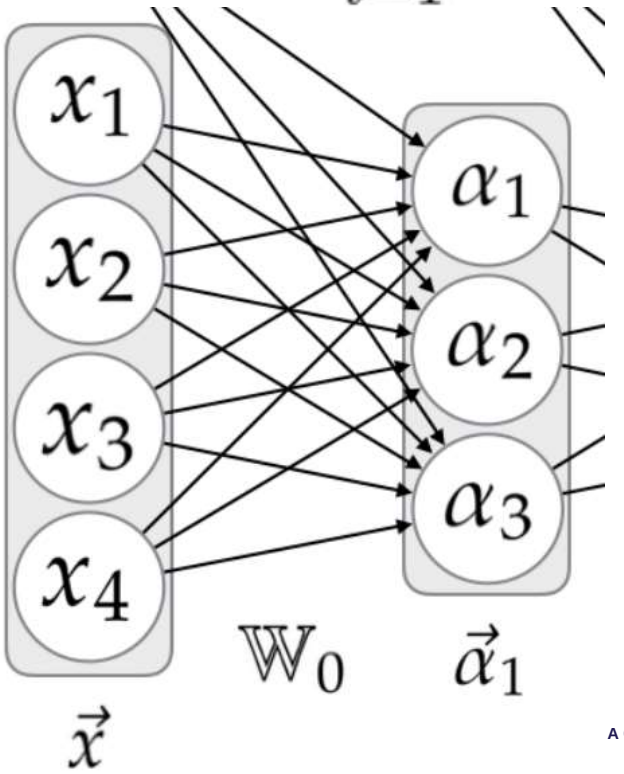
[Bjerva, 2017]

Inference in an FFNN



$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^{n} x_i y_i$$

- Same as in a perceptron!
- …but with layers
- Transformation from input (x) to activations (alpha)
- …and a non-linearity



$$\vec{\alpha}_1 = \boxed{\sigma}\left(\boxed{\mathbb{W}_0 \vec{x}}\right)$$

FFNNs: Behind the Scenes

- x, α_1, α_2, etc., are *vectors*
- W_0, W_1, etc. are *matrices*
- What is the dimensionality (or shape) of W_0?
- We know that:
  - x is in R^4
  - α_1 is in R^3
- So, W is a 4x3 matrix

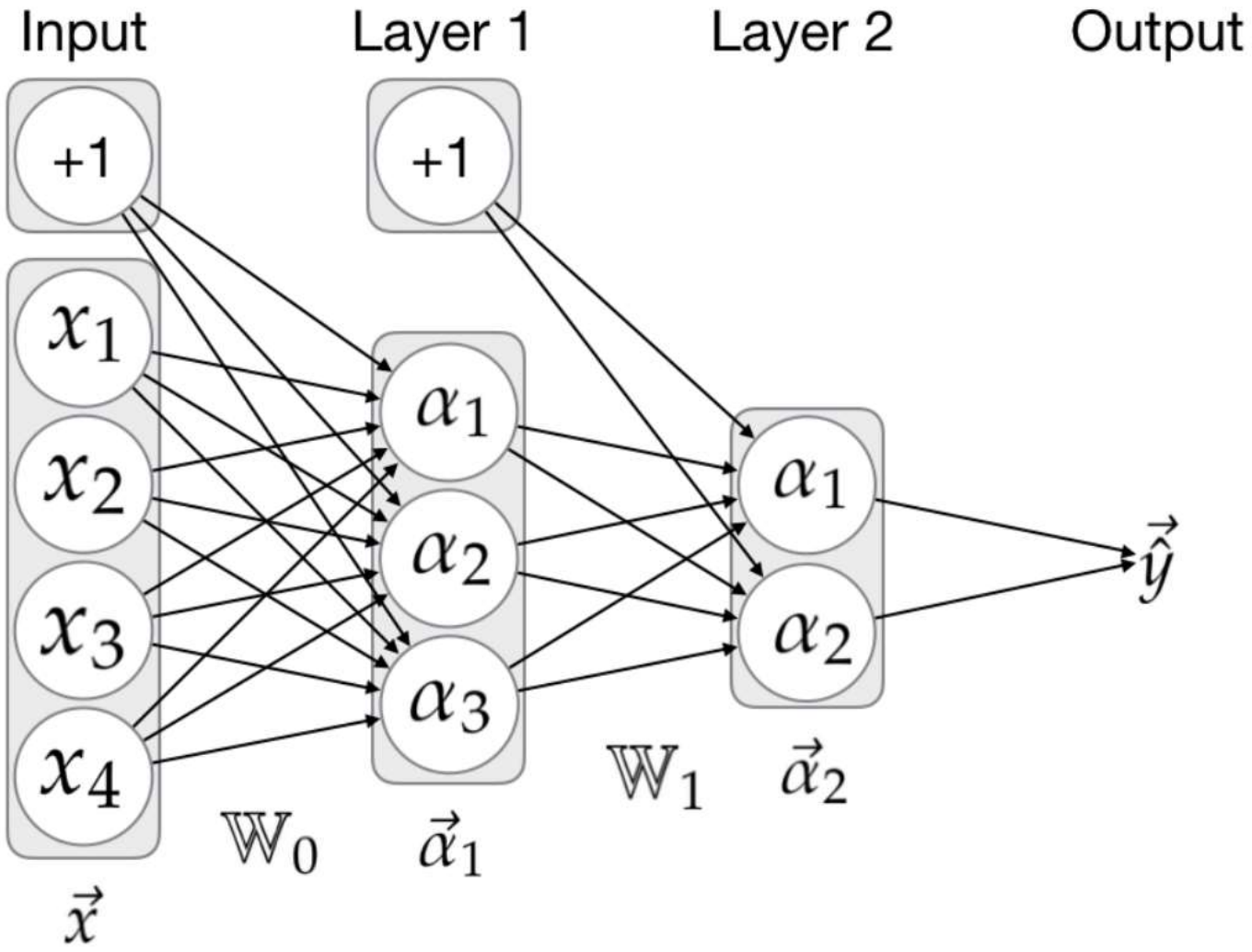


Figure 2.1: A basic Neural Network.

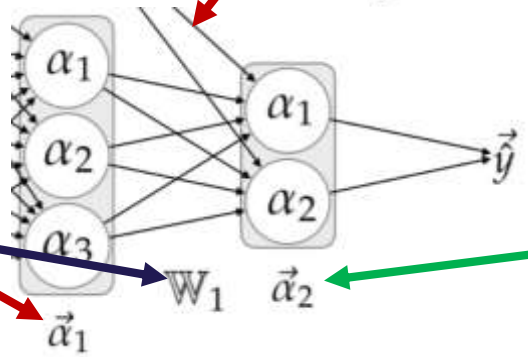
Do you remember Matrix-Vector Products?

$$A\mathbf{x} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \end{bmatrix}.$$

$$\mathbf{x} = (2, 1, 0)$$

$$A = \begin{bmatrix} 1 & -1 & 2 \\ 0 & -3 & 1 \end{bmatrix}$$

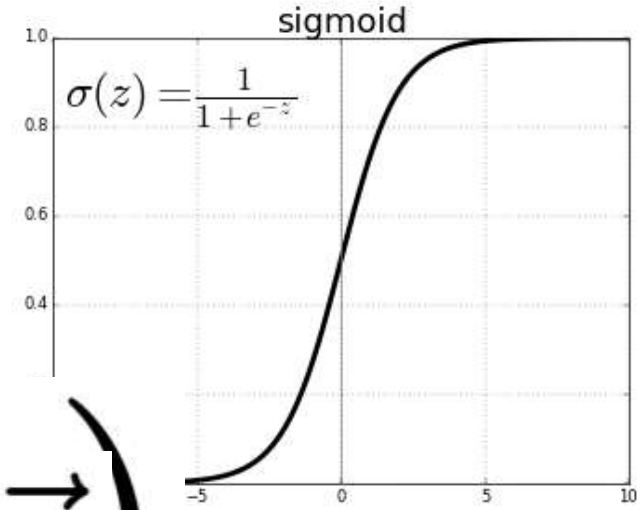
$$A\mathbf{x} = \begin{bmatrix} 1 & -1 & 2 \\ 0 & -3 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \quad \end{bmatrix} = \begin{bmatrix} \quad \end{bmatrix}.$$

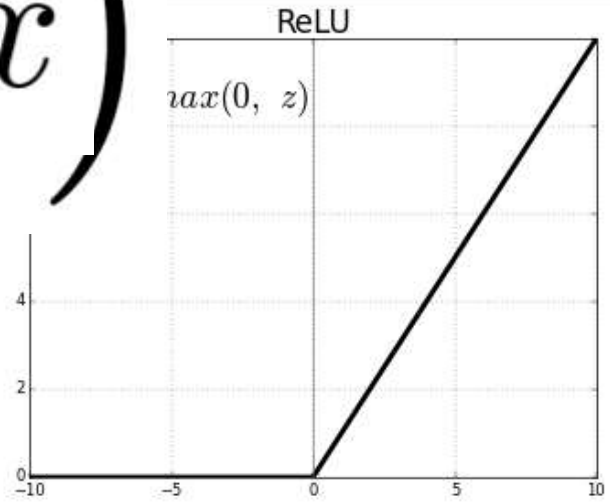


Activation Functions

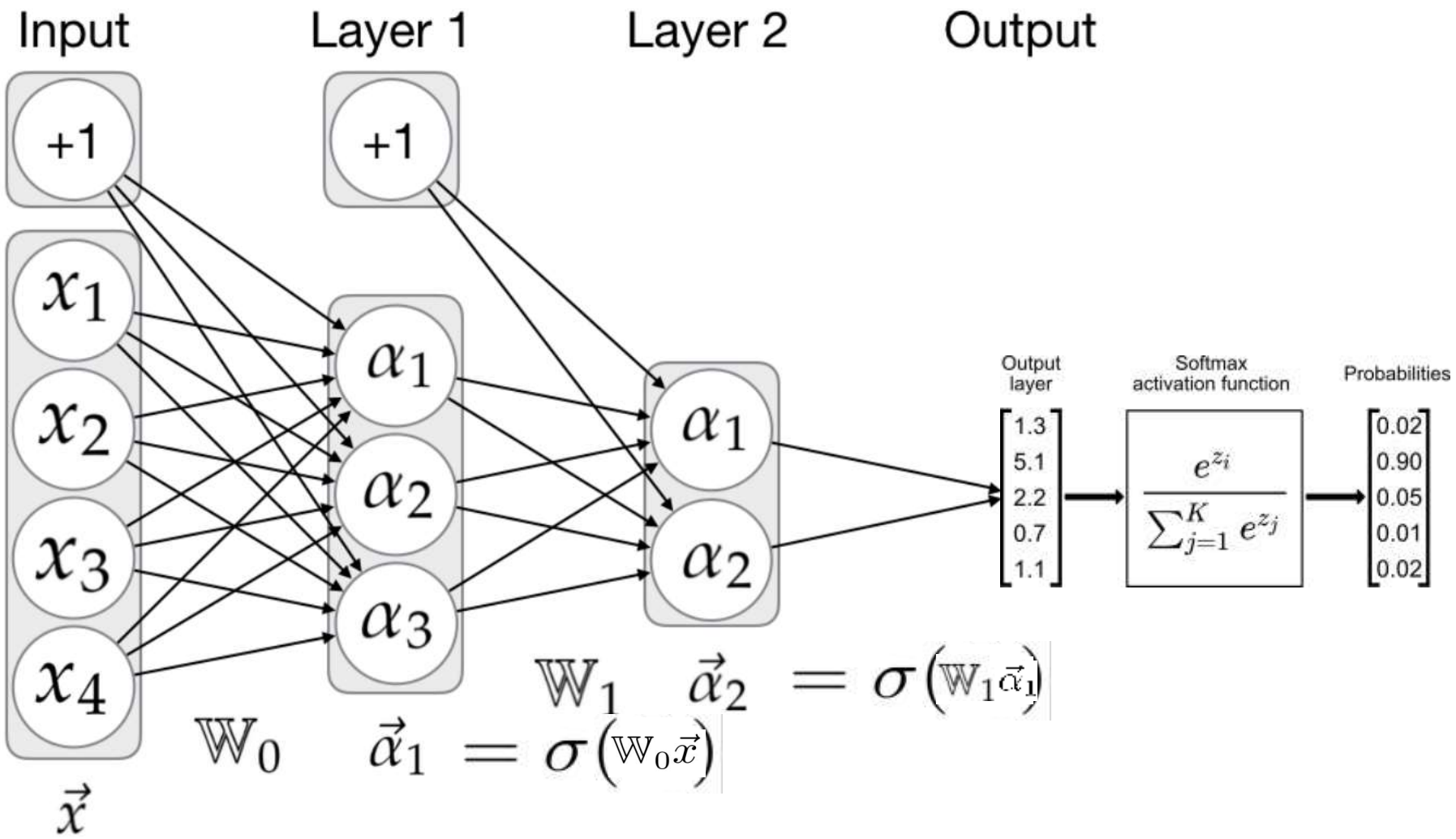
Name	Function
Logistic (ak	
Hyperbolic	
Rectified Linear Unit (ReLU)	$f(x) = \begin{cases} 0 & \text{for} \quad x < 0 \\ x & \text{for} \quad x \geq 0 \end{cases}$
Leaky ReLU	$f(x) = \begin{cases} 0.01x & \text{for} \quad x < 0 \\ x & \text{for} \quad x \geq 0 \end{cases}$
Softmax	$f(\vec{x})_i = \frac{e^{x_i}}{\sum_{k=1}^{K} e^{x_k}}$ for $i = 1, \ldots, K$

$$\vec{\alpha}_1 = \boxed{\sigma}\left(\mathbb{W}_0 \vec{x}\right)$$





Inference: Matrix-Multiplication + Activation



How is Learning Done?

- Gradient-based learning with an optimisation algorithm
  - Stochastic Gradient Descent
- Loss function in classification:
  - Usually Cross-entropy

$$L_{cross-entropy}(\vec{\hat{y}}, \vec{y}) = -\sum_i \vec{y}_i \log \vec{\hat{y}}_i$$

Implementation: What can we change in an FFNN?

- Architecture:
  - Layers
    - Number of layers, width (n neurons)
  - Activation functions
    - Relu, gelu,
  - Dropout between layers
- Training:
  - Optimisation algorithm
    - SGD, Adam
    - Learning rates
  - Batch size

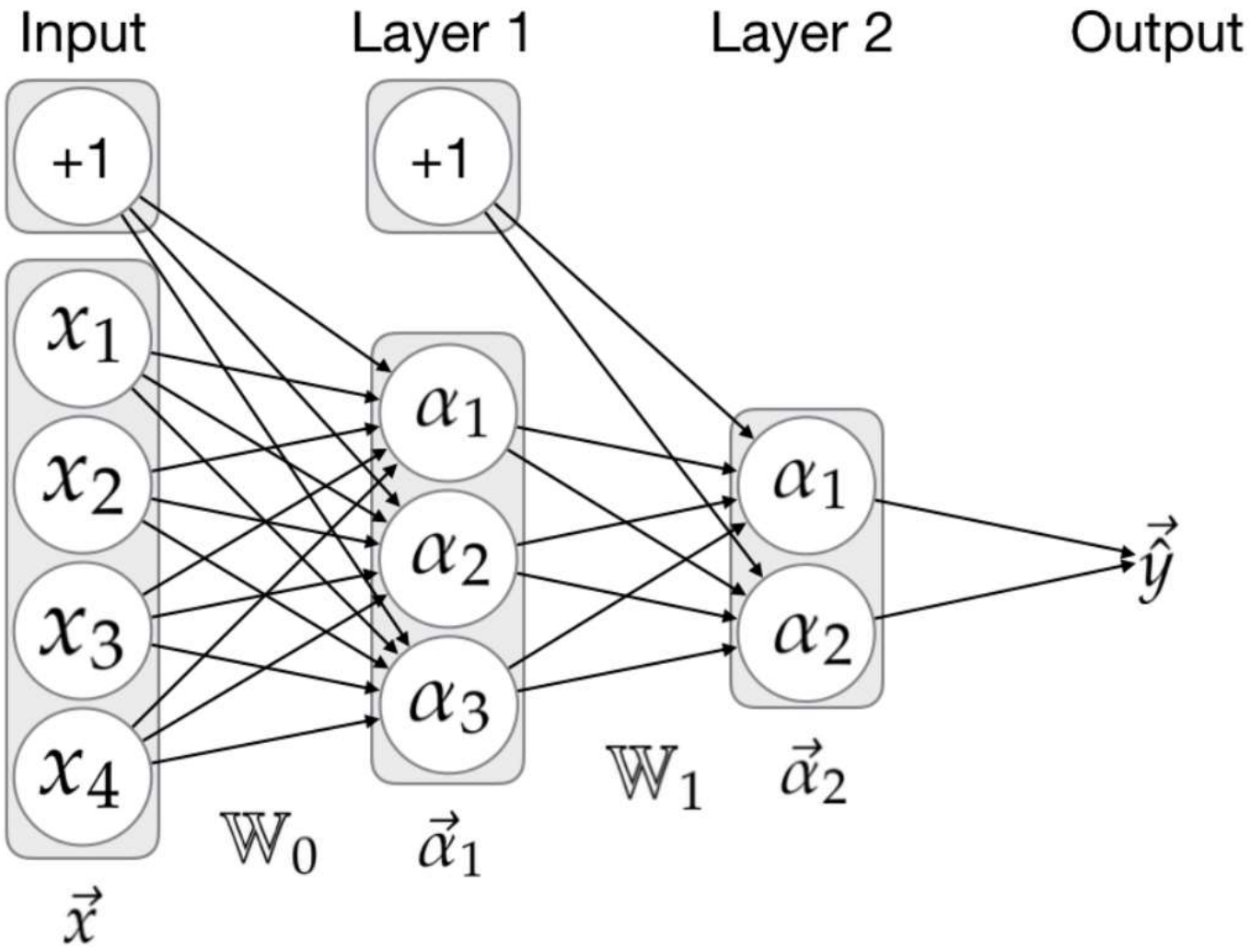


Figure 2.1: A basic Neural Network.

Notebook Time:
Implementation of FFNNs: Keras

- **Notebook:** tinyurl.com/mi-ffnn-notebook
- **First run the cells [1] to [4] <u>once</u>**
- **Try to get the best test accuracy!**
- **Input your accuracy in the form for a chance for □ ETERNAL GLORY □**
  - Unlimited number of submissions
- **Remember to report your *test* result (accuracy):** tinyurl.com/mi-leaderboard-ffnn
- Observations?
  - Is there an issue with the fact that we evaluate on the **test** set so many times, and report only the highest?

Wrap-up

- Perceptrons
  - Basics of NNs
  - Linear learner
- FFNNs
  - One step further
  - Can learn ”anything” (in theory)
- Implementation in Keras
- **Exercise: Evaluation Metrics**

Exercise:
Implementation of Evaluation Metrics

- **In the same notebook:**
  https://colab.research.google.com/drive/1iHAAaRaIY_4ublTlFBflQTOrCDtdqL_G?usp=sharing
- **Implement Precision, Recall, and F-score, and compare with your accuracy results**
- **Is the winner still the same?**

https://tinyurl.com/mi-e22-f5-ffnn

Evaluation Metrics

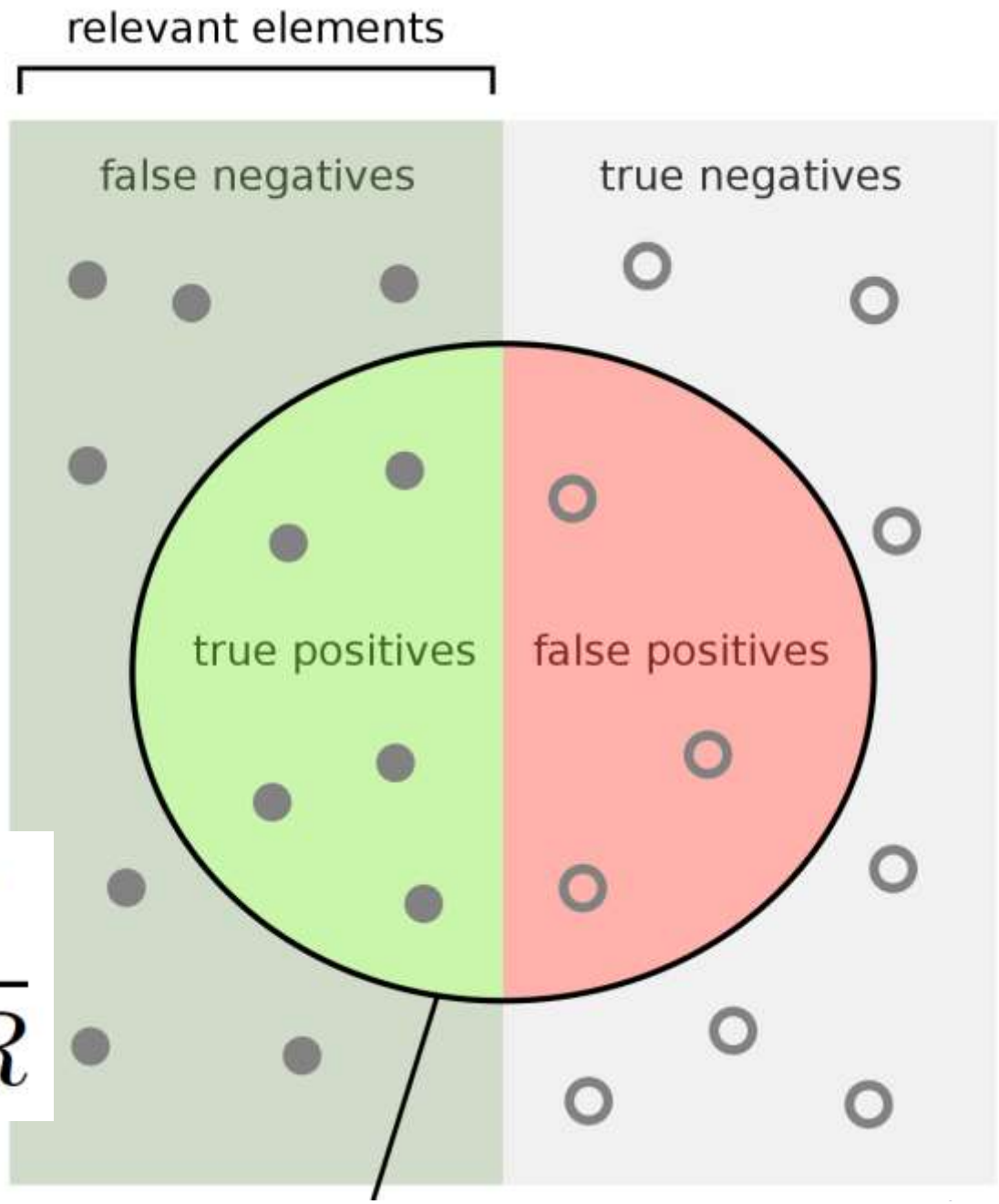
- Is accuracy always enough?
- Sometimes we care more about certain outcomes
- Sometimes data is very inbalanced

	Classifier predicted +	Classifier predicted −
Gold label is +	True positive (TP)	False negative (FN)
Gold label is −	False positive (FP)	True negative (TN)

Precision and Recall

- **Precision:**
  - How many of the retrieved entries are relevant
- **Recall:**
  - How many of the relevant entries are retrieved
- **F-score** weights the two, and is often a good indicator of performance
- Question: When is F-score more representative than accuracy?

$$P = \frac{TP}{TP + FP}$$

$$R = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2PR}{P + R}$$